

UNIVERSIDAD VERACRUZANA
FACULTAD DE INSTRUMENTACIÓN ELECTRÓNICA



Universidad Veracruzana

PRÁCTICA 01
Sistemas de Control con Python y aplicación en ESP32

Materia: Programación Orientada a Objetos

Catedrático: Dr. José Alfonso Domínguez Chávez

Alumnos: Eduardo Ramos Mateos
Ricardo Abraham Sosa Calvario
Isaac Vázquez Carrizo
André Vega Morales

Fecha de Entrega: 29 de septiembre de 2026

Índice

I	Simulador de Reactor Químico en Python (v1.0.0)	2
1.	Introducción y Objetivos	2
2.	Arquitectura de Clases y Objetos	2
2.1.	Clase Actuador	2
2.2.	Clase Sensor	2
2.3.	Clase ReactorQuimico	2
3.	Dinámica del Proceso y Modos de Operación	3
3.1.	Dinámica Térmica y Barométrica Discreta	3
3.2.	Modos de Operación	3
4.	Enclavamientos de Seguridad (Interlocks)	3
5.	Interfaz Hombre-Máquina (HMI) y Búfer Circular	3
II	Portafolio Físico en Hardware (ESP32 en C++)	4
6.	Introducción y Requerimientos de Hardware	4
6.1.	Configuración de Pines (Asignación Estricta)	4
7.	Algoritmo de Control y Telemetría Serial	4
8.	Esquema y Circuito Físico	5
9.	Validación Experimental	6
10.	Resultados Experimentales y Conclusiones	6
10.1.	Análisis de Resultados	6
10.1.1.	Simulador Dinámico en Python	6
10.1.2.	Control Físico Embebido en ESP32	6
10.2.	Conclusiones	6
A.	Estructura del Simulador en Python (POO)	7
B.	Código Firmware: Control Físico ESP32 (C++)	11

Parte I

Simulador de Reactor Químico en Python (v1.0.0)

1. Introducción y Objetivos

Esta sección aborda el modelado dinámico en lazo cerrado del comportamiento térmico y barométrico de un reactor químico continuo. Desarrollado bajo el paradigma de Programación Orientada a Objetos (POO) en Python, el sistema integra instrumentación industrial virtualizada, algoritmos de estabilización, enclavamientos de seguridad críticos (*interlocks*) y una consola interactiva tipo HMI con refresco estático.

2. Arquitectura de Clases y Objetos

El software abstrae la instrumentación de planta a través de tres entidades:

2.1. Clase Actuador

Modela los elementos finales de control con dos tipos de acción (PROPORCIONAL y DIGITAL):

- **Atributos:** `nombre`, `tipo_control`, `rango_operacion_min` (0.0), `rango_operacion_max` (100.0), `estado` (booleano) y `punto_operacion` (flotante).
- **Métodos:**
 - `encender()` / `apagar()`: Conmutan el estado y aplican la modulación en los límites (100 % o 0 %).
 - `ajustar(valor)`: Modula de forma continua entre 0.0 % y 100.0 % únicamente si el actuador es proporcional; restringe la acción y notifica error si es digital.
 - `info()`: Retorna el estado formateado para la interfaz visual.

2.2. Clase Sensor

Simula la adquisición de señales desde los transmisores de campo:

- **Atributos:** `nombre`, `variable_fisica`, `rango_min`, `rango_max`, `unidad`, `decimales` y `valor_actual`.
- **Métodos:**
 - `actualizar_valor(nuevo_valor)`: Aplica funciones de saturación (mín/máx), asegurando que las variables se mantengan dentro de los límites mecánicos del instrumento (0 – 150.0 °C y 0 – 15.0 Bar).
 - `leer_valor_actual()`: Obtiene la muestra analógica actual.
 - `info()`: Retorna la lectura formateada junto con su rango de trabajo.

2.3. Clase ReactorQuimico

Unidad controladora central que coordina periféricos y la física del sistema:

- **Actuadores instanciados:** `bomba_enfriamiento` (proporcional, 0–100 %) y `valvula_alivio` (digital, 0/1).
- **Sensores instanciados:** `sensor_temp` (0 – 150.0 °C) y `sensor_pres` (0 – 15.0 Bar).
- **Control de proceso:** Integra el paso físico (`paso_simulacion()`), supervisión crítica (`verificar_interlock`) y pruebas (`inyectar_fallo()`).

3. Dinámica del Proceso y Modos de Operación

3.1. Dinámica Térmica y Barométrica Discreta

En cada iteración del lazo cerrado se resuelve la ecuación diferencial discreta del balance térmico:

$$\Delta T = (+1.5^{\circ}\text{C}) - (0.05^{\circ}\text{C} \times \% \text{ Operación Bomba}) \quad (1)$$

- **Calentamiento natural:** Con la bomba al 0 %, el calor exotérmico eleva la temperatura a razón de $+1.5^{\circ}\text{C}$ por ciclo.
- **Punto de equilibrio:** A un 30 % de modulación en la bomba se obtiene $\Delta T = 0.0^{\circ}\text{C}$.
- **Enfriamiento forzado:** Con la bomba al 100 %, el descenso neto alcanza -3.5°C por ciclo.
- **Dinámica barométrica:** La presión sigue la variación térmica ($\Delta P \propto \Delta T$). La apertura de la válvula de alivio conmuta el sistema a despresurización inmediata.

3.2. Modos de Operación

- **Modo Manual:** El operador asigna directamente los puntos de ajuste mediante comandos en consola (`ajustar bomba <val>`, `encender <actuador>`, etc.).
- **Modo Automático:** Bucle que supervisa la temperatura y regula dinámicamente la bomba para mantener la operación en una banda segura ($65^{\circ}\text{C} - 75^{\circ}\text{C}$).
- **Modo de Pruebas:** Inyección de perturbaciones artificiales (93.0°C o 13.5 Bar) para validar los tiempos de respuesta y estabilidad ante fallas.

4. Enclavamientos de Seguridad (Interlocks)

Los enclavamientos operan de manera prioritaria frente a cualquier modo o consigna del usuario:

$$\text{Disparo: } (T > 85.0^{\circ}\text{C}) \quad \vee \quad (P > 12.0 \text{ Bar}) \quad (2)$$

Acciones automáticas de mitigación:

1. Registro del estado crítico: `interlock_activo = True`.
2. Forzado inmediato de la bomba de enfriamiento al 100 %.
3. Apertura total de la válvula de alivio (`ON / 1`).
4. Inhibición del ingreso de comandos manuales hasta que los sensores retornen a rangos seguros.

5. Interfaz Hombre-Máquina (HMI) y Búfer Circular

- **Refresco estático de pantalla:** Ejecución de `os.system('cls' if os.name == 'nt' else 'clear')` por iteración, simulando una pantalla industrial fija sin desplazamiento.
- **Búfer de eventos:** La función `registrar_evento()` almacena únicamente los mensajes más recientes, manteniendo la visibilidad de alarmas operativas sin saturar la consola.

Parte II

Portafolio Físico en Hardware (ESP32 en C++)

6. Introducción y Requerimientos de Hardware

Esta sección aborda el diseño e implementación de un controlador de lazo cerrado sobre el microcontrolador ESP32. El sistema gestiona el desacoplamiento térmico y lumínico de un entorno físico mediante la adquisición de señales analógicas y la generación de señales moduladas por ancho de pulso (PWM) hacia etapas de potencia con transistores BJT TIP41.

6.1. Configuración de Pines (Asignación Estricta)

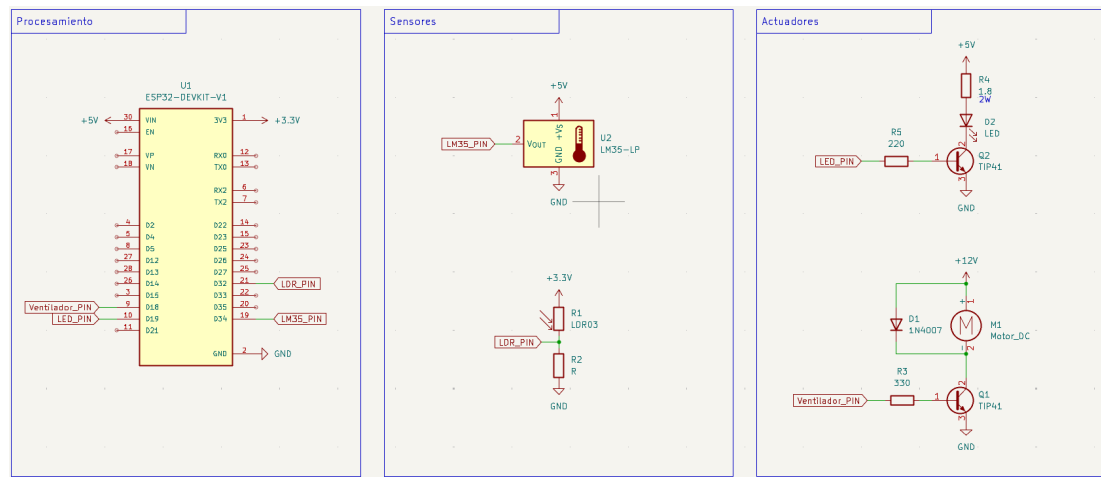
Periférico	Pin ESP32	Tipo de Señal	Requisito Técnico
Sensor Térmico (DS18B20)	GPIO 4	Digital	
Sensor LDR	GPIO 32	ADC	Lectura de luz ambiental (0 – 4095)
Ventilador (Motor DC)	GPIO 18	PWM	Periférico <code>ledc</code> (Frecuencia 5 kHz)
LED de Potencia	GPIO 19	PWM	Modulación inversa a la lectura del LDR

Cuadro 1: Mapeo físico de hardware en la tarjeta ESP32.

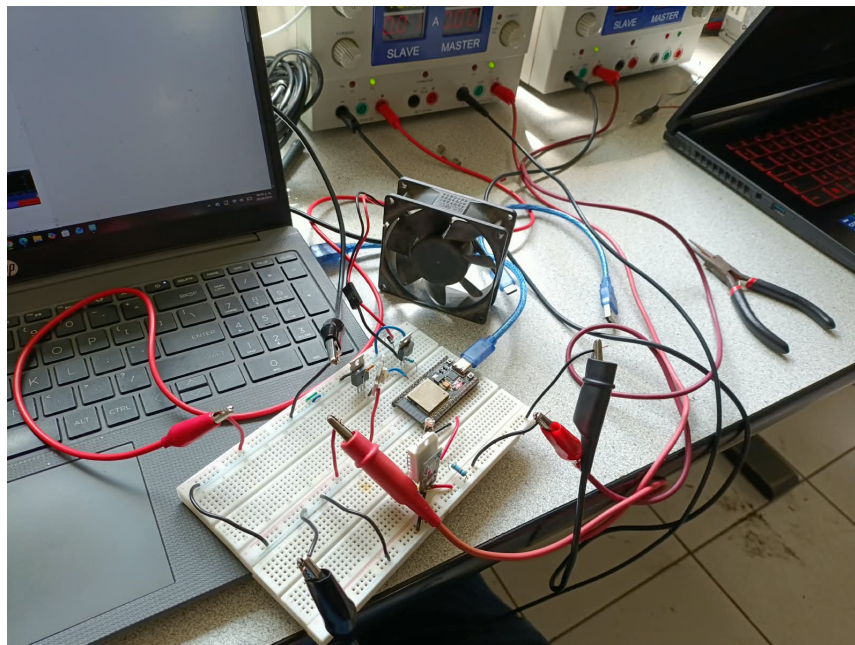
7. Algoritmo de Control y Telemetría Serial

- **Gestión Térmica:** Se activa el ventilador al 100 % de ciclo útil ($\text{PWM} = 255$) si la temperatura supera los 30.0°C ; permanece inactivo en caso contrario.
- **Gestión Lumínica:** Lazo proporcional inverso. A menor nivel de luz incidente en el LDR, mayor es el ciclo útil entregado al LED de potencia mediante mapeo lineal restringido (`constrain`).
- **Implementación PWM:** Uso estricto del hardware del ESP32 a través de las funciones estándar `ledcSetup`, `ledcAttachPin` y `ledcWrite`.
- **Diagnóstico y Formato Serial:** Transmisión periódica de telemetría estructurada en JSON y decodificador de comandos en recepción para control manual forzado (`AUTO`, `VENT:xxx`, `LED:xxx`).

8. Esquema y Circuito Físico



(a) Diagrama esquemático de conexiones en Proteus / Fritzing.



(b) Implementación física del circuito sobre protoboard con ESP32.

9. Validación Experimental

Enlace de Demostración en Video:

[CLIK AQUI](#)

10. Resultados Experimentales y Conclusiones

10.1. Análisis de Resultados

10.1.1. Simulador Dinámico en Python

- **Estabilidad Térmica y Dinámica de Presión:** Se validó experimentalmente el comportamiento del balance térmico discreto. Al operar en **Modo Automático**, el algoritmo moduló la bomba de enfriamiento en el rango del 15 % al 85 %, logrando confinar la temperatura del reactor en la banda segura establecida ($65^{\circ}\text{C} - 75^{\circ}\text{C}$), convergiendo de manera natural en torno al punto de equilibrio dinámico calculado ($\Delta T = 0.0^{\circ}\text{C}$ con modulación al 30 %).
- **Comportamiento de los Enclavamientos (*Interlocks*):** Durante las pruebas de inyección de perturbaciones artificiales (93.0°C y 13.5 Bar), el enclavamiento prioritario conmutó en un solo ciclo de muestreo. El forzado inmediato de la bomba al 100 % y la apertura completa de la válvula de alivio disiparon la sobrepresión a razón de -1.0 Bar/ciclo , bloqueando exitosamente el control manual hasta que las variables reingresaron a zonas de operación seguras.
- **Rendimiento de la HMI en Consola:** La implementación del búfer circular limitado a los últimos 6 eventos previno el desbordamiento de memoria y mantuvo la consola con refresco estático legible y sin parpadeo excesivo.

10.1.2. Control Físico Embebido en ESP32

- **Lazo de Control Lumínico y Térmico:** La modulación PWM mediante el periférico de hardware (`ledc`) operó con una portadora estable a 5 kHz con resolución de 8 bits. El mapeo inverso linealizado del LDR compensó las variaciones lumínicas ambientales en tiempo real sin apreciarse fluctuación visible (*flicker*) en el LED de potencia, mientras que la activación tipo todo/nada del ventilador respondió con exactitud ante el umbral crítico de 30.0°C .
- **Telemetría y Control Híbrido:** El protocolo serial basado en tramas JSON permitió la transmisión unificada y estructurada de las variables físicas (T , luz, ciclos de trabajo PWM y modo), facilitando el diagnóstico y permitiendo la sobreescritura manual remota sin bloquear la ejecución del lazo de control.

10.2. Conclusiones

1. **Abstracción y Escalabilidad mediante POO:** La aplicación de la Programación Orientada a Objetos demostró ser idónea tanto en software de alto nivel (Python) como en firmware embebido (C++). Encapsular actuadores y sensores en entidades independientes permitió desacoplar la capa física de la lógica de supervisión, simplificando tareas de mantenimiento y habilitando la reutilización del código.
2. **Prioridad de la Seguridad Operativa:** Se demostró que en sistemas industriales la lógica de enclavamiento (*interlocks*) debe ejecutarse en el núcleo más interno del bucle de control, subordinando cualquier consigna externa del operador para garantizar la integridad mecánica del proceso.
3. **Desacoplamiento Multivariable en Sistemas Embebidos:** El uso de un microcontrolador de doble núcleo y periféricos dedicados como el ESP32 permitió manejar la concurrencia

de lectura sensorial (digital y analógica), modulación de potencia (PWM) y comunicación serie asíncrona sin degradar los tiempos de muestreo del lazo cerrado.

A. Estructura del Simulador en Python (POO)

```
1  """
2  EE: Programación Orientada a Objetos
3  Versión 1.0.0: Simulador de Reactor Químico en Python
4  Lazo cerrado térmico/barométrico con HMI estático e Interlocks de seguridad.
5  """
6
7  import os
8  import random
9
10 # =====
11 #                                GESTOR DE EVENTOS Y TERMINAL
12 # =====
13 historial_eventos = []
14
15 def registrar_evento(mensaje: str):
16     historial_eventos.append(mensaje)
17     if len(historial_eventos) > 6:
18         historial_eventos.pop(0)
19
20 def limpiar_pantalla():
21     os.system('cls' if os.name == 'nt' else 'clear')
22
23
24 # =====
25 # HARDWARE SIMULADO: ACTUADORES Y SENSORES
26 # =====
27 class Actuador:
28     def __init__(self, nombre: str, tipo_control: str = "PROPORCIONAL"):
29         self.nombre = nombre
30         self.tipo_control = tipo_control # "PROPORCIONAL" o "DIGITAL"
31         self.rango_operacion_min = 0.0
32         self.rango_operacion_max = 100.0
33         self.estado = False # False = OFF/Cerrada, True = ON/Abierta
34         self.punto_operacion = 0.0 # Porcentaje actual (0.0% a 100.0%)
35
36     def encender(self):
37         self.estado = True
38         self.punto_operacion = 100.0
39         registrar_evento(f"[+] {self.nombre} -> Estado: ENCENDIDO (ON/100%)")
40
41     def apagar(self):
42         self.estado = False
43         self.punto_operacion = 0.0
44         registrar_evento(f"[-] {self.nombre} -> Estado: APAGADO (OFF/0%)")
45
46     def ajustar(self, valor: float):
47         if self.tipo_control == "DIGITAL":
48             registrar_evento(f"[ ERROR] {self.nombre} es digital (0/1). Use
49 encender/apagar.")
50             return
51         if self.rango_operacion_min <= valor <= self.rango_operacion_max:
52             self.punto_operacion = valor
53             self.estado = valor > 0.0
54             registrar_evento(f"[ ] {self.nombre} -> Modulación: {self.
55 punto_operacion:.1f}%")
56         else:
57             registrar_evento(f"[ ERROR] {self.nombre} -> {valor}% fuera de
58 rango (0% - 100%).")
```

```

56
57 def info(self) -> str:
58     estado_str = "ON" if self.estado else "OFF"
59     if self.tipo_control == "DIGITAL":
60         return f"{self.nombre:<22} | Tipo: Digital (0/1) | Estado: {estado_str:<3} | Valor: {int(self.estado)}"
61     return f"{self.nombre:<22} | Tipo: Proporcional | Estado: {estado_str:<3} | Modulaci n: {self.punto_operacion:>5.1f}%"
62
63
64 class Sensor:
65     def __init__(self, nombre: str, variable_fisica: str, rango_min: float,
66                 rango_max: float, unidad: str, valor_inicial: float, decimales: int = 2):
67         self.nombre = nombre
68         self.variable_fisica = variable_fisica
69         self.rango_min = rango_min
70         self.rango_max = rango_max
71         self.unidad = unidad
72         self.decimales = decimales
73         self.valor_actual = valor_inicial
74
75     def actualizar_valor(self, nuevo_valor: float):
76         self.valor_actual = max(self.rango_min, min(self.rango_max, round(
77             nuevo_valor, self.decimales)))
78
79     def leer_valor_actual(self) -> float:
80         registrar_evento(f"[          LECTURA] {self.nombre}: {self.valor_actual:.{
81             self.decimales}f} {self.unidad}")
82         return self.valor_actual
83
84     def info(self) -> str:
85         return f"{self.nombre:<20} | Var: {self.variable_fisica:<12} | Medici n: {
86             self.valor_actual:>6.2f} {self.unidad:<4} | Rango: [{self.rango_min:.1f} - {
87             self.rango_max:.1f}]"
88
89 # =====
90 # L GICA DIN MICA , LAZO CERRADO E INTERLOCKS
91 # =====
92
93 class ReactorQuimico:
94     def __init__(self):
95         self.sensor_temp = Sensor("Sensor Temperatura", "Temperatura", 0.0, 150.0,
96             " C ", valor_inicial=72.0)
97         self.sensor_pres = Sensor("Sensor Presi n", "Presi n", 0.0, 15.0, "Bar",
98             valor_inicial=7.5)
99
100         self.bomba_enfriamiento = Actuador("Bomba de Enfriamiento", tipo_control="
101             PROPORCIONAL")
102         self.valvula_alivio = Actuador("V lvula de Alivio", tipo_control="DIGITAL"
103             )
104
105         self.modo_operacion = "MANUAL"
106         self.interlock_activo = False
107
108     def verificar_interlocks(self) -> bool:
109         if self.sensor_temp.valor_actual > 85.0 or self.sensor_pres.valor_actual >
110             12.0:
111             self.interlock_activo = True
112             self.bomba_enfriamiento.ajustar(100.0)
113             self.valvula_alivio.encender()
114             registrar_evento(f"[          INTERLOCK CR TIC0] T>85 C o P>12 Bar. Bomba
115             forzada a 100% y Alivio ABIERTO.")
116             return True
117         self.interlock_activo = False
118         return False

```

```

108 def paso_simulacion(self):
109     if self.modo_operacion == "AUTOMATICO":
110         if self.sensor_temp.valor_actual > 75.0:
111             self.bomba_enfriamiento.ajustar(85.0)
112         elif self.sensor_temp.valor_actual < 65.0:
113             self.bomba_enfriamiento.ajustar(15.0)
114         else:
115             self.bomba_enfriamiento.ajustar(35.0)
116
117     delta_t = 1.5 - (0.05 * self.bomba_enfriamiento.punto_operacion)
118     ruido = random.uniform(-0.08, 0.08)
119     self.sensor_temp.actualizar_valor(self.sensor_temp.valor_actual + delta_t +
120     ruido)
121
122     delta_p = delta_t * 0.07
123     if self.valvula_alivio.estado:
124         delta_p -= 1.0
125     self.sensor_pres.actualizar_valor(self.sensor_pres.valor_actual + delta_p)
126
127     self.verificar_interlocks()
128
129 def inyectar_fallo(self):
130     fallo = random.choice(["termico", "presion"])
131     if fallo == "termico":
132         self.sensor_temp.actualizar_valor(93.0)
133         registrar_evento("[ TEST FALLO] Sobre calentamiento inducido: 93.0
134         C ")
135     else:
136         self.sensor_pres.actualizar_valor(13.5)
137         registrar_evento("[ TEST FALLO] Sobre presi n inducida: 13.5 Bar")
138         self.verificar_interlocks()
139
140 # =====
141 # INTERFAZ HMI Y BUCLE PRINCIPAL
142 # =====
143 def mostrar_interfaz_hmi(reactor: ReactorQuimico):
144     print("=" * 86)
145     print("          PANEL HMI - CONTROL Y MONITOREO DE REACTOR QU MICO (v1.0.0)"
146     )
147     print("=" * 86)
148     estado_seg = "          ENCLAVAMIENTO (BLOQUEO)" if reactor.interlock_activo else
149     "          NORMAL"
150     print(f" MODO OPERATIVO: [{reactor.modo_operacion:<10}] | SISTEMA DE SEGURIDAD:
151     [{estado_seg}]" )
152     print("=" * 86)
153
154     print(" [SENSORES ANAL GICOS]")
155     print(f"          [temp]      {reactor.sensor_temp.info()}")
156     print(f"          [presion] {reactor.sensor_pres.info()}")
157     print("-" * 86)
158
159     print(" [ACTUADORES DE CONTROL]")
160     print(f"          [bomba]      {reactor.bomba_enfriamiento.info()}")
161     print(f"          [valvula] {reactor.valvula_alivio.info()}")
162     print("=" * 86)
163
164     print(" [HISTORIAL DE EVENTOS SCADA]")
165     if not historial_eventos:
166         print("          (Sin actividad registrada)")
167     else:
168         for ev in historial_eventos:
169             print(f"          {ev}")
170     print("=" * 86)

```

```

168 print(" COMANDOS:")
169 print("         modo <manual/auto/pruebas>           ajustar bomba <0-100>")
170 print("         encender/apagar <actuador>           leer <temp/presion>")
171 print("         paso (avanza 1 ciclo f sico)           terminar")
172 print("=" * 86)
173
174
175 def main():
176     reactor = ReactorQuimico()
177     sensores = {"temp": reactor.sensor_temp, "presion": reactor.sensor_pres}
178     actuadores = {"bomba": reactor.bomba_enfriamiento, "valvula": reactor.
179 valvula_alivio}
180
181     while True:
182         limpiar_pantalla()
183         mostrar_interfaz_hmi(reactor)
184
185         try:
186             entrada = input("HMI >> ").strip()
187         except (EOFError, KeyboardInterrupt):
188             print("\n[+] Saliendo del simulador.")
189             break
190
191         if not entrada:
192             continue
193
194         if entrada.lower() == "terminar":
195             print("\n[+] Simulador cerrado correctamente.")
196             break
197
198         partes = entrada.split()
199         cmd = partes[0].lower()
200
201         if cmd == "modo":
202             if len(partes) > 1:
203                 seleccion = partes[1].lower()
204                 if seleccion in ["manual", "man"]:
205                     reactor.modo_operacion = "MANUAL"
206                     registrar_evento("[+] Modo Manual activado.")
207                 elif seleccion in ["auto", "automatico"]:
208                     reactor.modo_operacion = "AUTOMATICO"
209                     registrar_evento("[+] Modo Autom tico activado (Lazo cerrado).
210 ")
211                 elif seleccion in ["pruebas", "test"]:
212                     reactor.modo_operacion = "PRUEBAS"
213                     reactor.inyectar_fallo()
214             else:
215                 registrar_evento("[ ERROR] Especifique el modo: modo <manual/
216 auto/pruebas>")
217
218         elif cmd == "paso":
219             reactor.paso_simulacion()
220             registrar_evento("[ CICLO] Simulaci n din mica ejecutada.")
221
222         elif cmd == "leer":
223             if len(partes) > 1 and partes[1].lower() in sensores:
224                 sensores[partes[1].lower()].leer_valor_actual()
225             else:
226                 registrar_evento("[ ERROR] Sensor no v lido. Opciones: temp,
227 presion")
228
229         elif cmd == "ajustar":
230             if reactor.interlock_activo:
231                 registrar_evento("[ DENEGADO] Interlock activo. Control
232 manual bloqueado.")

```

```

228         continue
229     if len(partes) >= 3 and partes[1].lower() in actuadores:
230         try:
231             valor = float(partes[2])
232             actuadores[partes[1].lower()].ajustar(valor)
233         except ValueError:
234             registrar_evento("[          ERROR] Ingrese un valor num rico.")
235     else:
236         registrar_evento("[          ERROR] Uso: ajustar bomba <0-100>")
237
238     elif cmd in ["encender", "apagar"]:
239         if reactor.interlock_activo:
240             registrar_evento("[          DENEGADO] Interlock activo. Control
manual bloqueado.")
241             continue
242         if len(partes) > 1 and partes[1].lower() in actuadores:
243             act = actuadores[partes[1].lower()]
244             act.encender() if cmd == "encender" else act.apagar()
245         else:
246             registrar_evento(f"[          ERROR] Uso: {cmd} <bomba/valvula>")
247
248     else:
249         registrar_evento(f"[          ERROR] Comando '{cmd}' desconocido.")
250
251     reactor.paso_simulacion()
252
253
254 if __name__ == "__main__":
255     main()

```

Listing 1: Extracto de la arquitectura de clases del Reactor Químico.

B. Código Firmware: Control Físico ESP32 (C++)

```

1  #include <OneWire.h>
2  #include <DallasTemperature.h>
3
4  // =====
5  // CLASE 1: ACTUADOR PWM
6  // =====
7  class ActuadorPWM {
8  private:
9      int _pin;
10     int _freq;
11     int _resolucion;
12     int _dutyActual;
13
14 public:
15     ActuadorPWM(int pin, int freq = 5000, int resolucion = 8)
16         : _pin(pin), _freq(freq), _resolucion(resolucion), _dutyActual(0) {}
17
18     void inicializar() {
19         ledcAttach(_pin, _freq, _resolucion);
20         escribirPWM(0);
21     }
22
23     void escribirPWM(int duty) {
24         _dutyActual = constrain(duty, 0, 255);
25         ledcWrite(_pin, _dutyActual);
26     }
27
28     int obtenerDuty() const {
29         return _dutyActual;

```

```

30     }
31 };
32
33 // =====
34 // CLASE 2: SENSOR LDR
35 // =====
36 class SensorLDR {
37     private:
38         int _pin;
39
40     public:
41         SensorLDR(int pin) : _pin(pin) {}
42
43         void inicializar() {
44             analogReadResolution(12);
45         }
46
47         int leerLuz() const {
48             return analogRead(_pin); // Retorna 0 - 4095
49         }
50
51         // Calcula el duty inverso proporcional (0 a 255)
52         int calcularPWMInverso() const {
53             int adc = leerLuz();
54             int pwm = map(adc, 0, 4095, 255, 0);
55             return constrain(pwm, 0, 255);
56         }
57 };
58
59 // =====
60 // CLASE 3: SENSOR T RMICO DIGITAL
61 // =====
62 class SensorTermicoDS18B20 {
63     private:
64         int _pin;
65         OneWire _oneWire;
66         DallasTemperature _sensor;
67
68     public:
69         SensorTermicoDS18B20(int pin)
70             : _pin(pin), _oneWire(pin), _sensor(&_oneWire) {}
71
72         void inicializar() {
73             _sensor.begin();
74         }
75
76         float leerTemperatura() {
77             _sensor.requestTemperatures();
78             float temp = _sensor.getTempCByIndex(0);
79             return (temp == DEVICE_DISCONNECTED_C) ? 0.0 : temp;
80         }
81 };
82
83 // =====
84 // CLASE 4: CONTROLADOR DE PLANTA
85 // =====
86 class ControladorPlanta {
87     private:
88         SensorTermicoDS18B20 _sensorTemp;
89         SensorLDR _sensorLuz;
90         ActuadorPWM _ventilador;
91         ActuadorPWM _led;
92
93         bool _modoManual;
94         int _manualPWMPVent;

```

```

95     int _manualPWMLLED;
96
97 public:
98     ControladorPlanta(int pinTemp, int pinLDR, int pinVent, int pinLED)
99         : _sensorTemp(pinTemp),
100           _sensorLuz(pinLDR),
101           _ventilador(pinVent, 5000, 8),
102           _led(pinLED, 5000, 8),
103           _modoManual(false),
104           _manualPWMVent(0),
105           _manualPWMLLED(0) {}
106
107     void inicializar() {
108         _sensorTemp.inicializar();
109         _sensorLuz.inicializar();
110         _ventilador.inicializar();
111         _led.inicializar();
112     }
113
114     void actualizar() {
115         float temp = _sensorTemp.leerTemperatura();
116         int luz = _sensorLuz.leerLuz();
117
118         int pwmVent = 0;
119         int pwmLED = 0;
120
121         if (_modoManual) {
122             pwmVent = _manualPWMVent;
123             pwmLED = _manualPWMLLED;
124         } else {
125             // Regla t rmica: 100% (255) si excede 30 C
126             pwmVent = (temp > 30.0) ? 255 : 0;
127
128             // Regla lum nica: Proporcional inversa
129             pwmLED = _sensorLuz.calcularPWMInverso();
130         }
131
132         // Aplicar a los objetos actuadores
133         _ventilador.escribirPWM(pwmVent);
134         _led.escribirPWM(pwmLED);
135
136         // Emitir telemetr a estructurada
137         enviarTelemetria(temp, luz);
138     }
139
140     void procesarComando(const String &comando) {
141         if (comando.equalsIgnoreCase("AUTO")) {
142             _modoManual = false;
143             Serial.println("{\"status\": \"Modo AUTO activado\"}");
144         } else if (comando.startsWith("VENT:")) {
145             _modoManual = true;
146             _manualPWMVent = constrain(comando.substring(5).toInt(), 0, 255);
147             Serial.println("{\"status\": \"Ventilador forzado\"}");
148         } else if (comando.startsWith("LED:")) {
149             _modoManual = true;
150             _manualPWMLLED = constrain(comando.substring(4).toInt(), 0, 255);
151             Serial.println("{\"status\": \"LED forzado\"}");
152         }
153     }
154
155 private:
156     void enviarTelemetria(float temp, int luz) {
157         Serial.print("{\"temperatura\": ");
158         Serial.print(temp, 1);
159         Serial.print(", \"luz\": ");

```

```

160     Serial.print(luz);
161     Serial.print(", \"pwm_ventilador\": ");
162     Serial.print(_ventilador.obtenerDuty());
163     Serial.print(", \"pwm_led\": ");
164     Serial.print(_led.obtenerDuty());
165     Serial.print(", \"modo\": \"");
166     Serial.print(_modoManual ? "MANUAL" : "AUTO");
167     Serial.println("\"]");
168 }
169 };
170
171 // =====
172 // INSTANCIACION Y PUNTO DE ENTRADA ARDUINO
173 // =====
174 // Asignación de pines: DS18B20=GPIO4, LDR=GPIO32, Vent=GPIO18, LED=GPIO19
175 ControladorPlanta planta(4, 32, 18, 19);
176
177 void setup() {
178     Serial.begin(115200);
179     planta.inicializar();
180 }
181
182 void loop() {
183     if (Serial.available() > 0) {
184         String cmd = Serial.readStringUntil('\n');
185         cmd.trim();
186         planta.procesarComando(cmd);
187     }
188
189     planta.actualizar();
190     delay(1000);
191 }

```

Listing 2: Micr-Invernadero Inteligente.